PORTFOLIO · 2026 · BACKEND ENGINEER

서비스를 만들고, 병목을 찾고,
운영 가능한 형태로 개선합니다.

김범수

Java · Spring Boot · Python · PostgreSQL · AWS

bumsookim333@naver.com · +82 10-4945-8463

github.com/BrianKim913 · medium.com/@bumsookim333

SELECTED WORK

세 개의 프로젝트, 세 개의 역할

혼자 기획부터 운영까지 맡은 서비스, 팀의 인프라를 책임진 프로젝트, 제한된 자원으로 겨룬 성능 대회입니다.

01

devport.kr

AI 트렌드·테크 뉴스·위키 플랫폼

1인 Full-Stack · 2025.12 – 운영 중

Native Image · PGO · RAG · Terraform

02

Looper

AI 회의록·일정 관리 서비스

DevOps · 6인 팀 · 2025.04 – 2025.08

kubeadm · AWS+GCP · ArgoCD · OpenTelemetry

03

반갈죽

실시간 채팅 부하테스트 대회

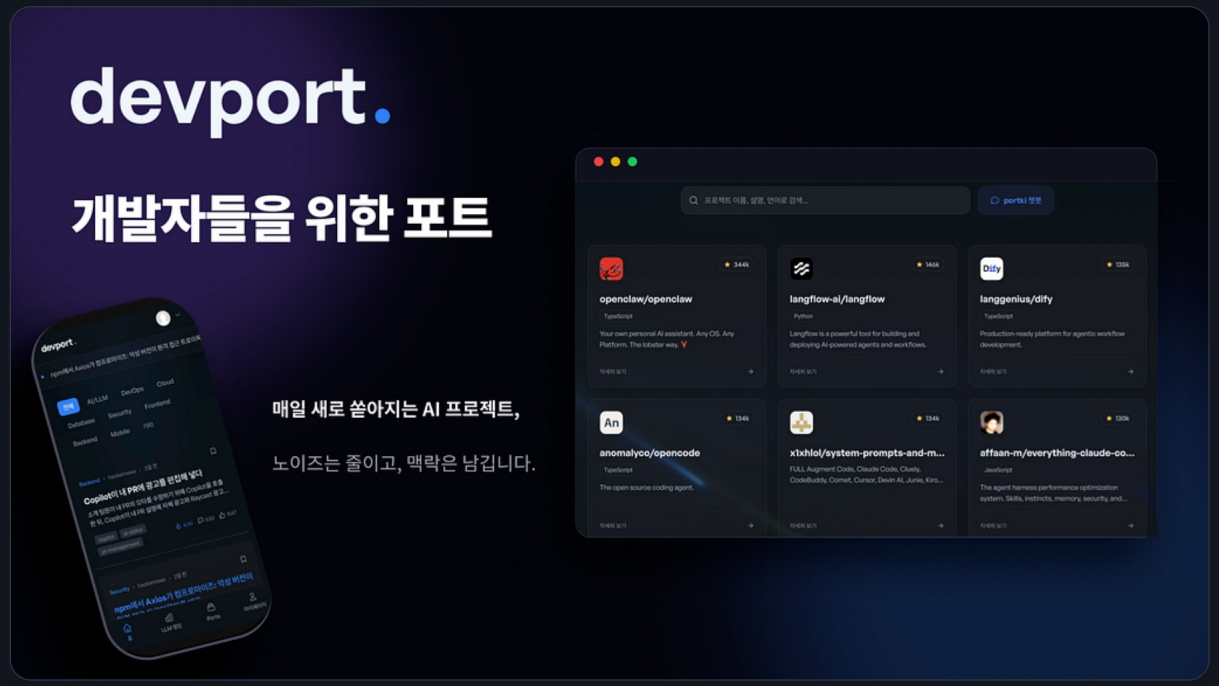
Backend · 2인 팀 · 2025.07

3위 / 22팀 · Redis Cluster · Socket.IO

01 · DEVPORT.KR

흩어진 기술 정보를 하나의 서비스로

해외 개발 뉴스와 GitHub 트렌드를 LLM으로 한국어 요약·분류하고, 프로젝트 위키와 RAG 챗봇을 제공합니다.



역할

1인 Full-Stack — 기획, Frontend, Backend, Crawler, AWS 인프라(IaC)

기간

2025.12 – 현재 운영 중

스택

Spring Boot 4 · GraalVM · Python · React · Terraform · PostgreSQL · Redis

01 · DEVPORT — DATA PIPELINE

수집부터 한국어 콘텐츠 제공까지

7개 소스의 콘텐츠를 비동기로 수집하고, LLM 요약·분류를 거쳐 API로 제공합니다.

STEP 1
멀티소스
Dev.to · Hashnode · Reddit · Hacker News · GitHub 등

→

STEP 2
병렬 수집 (크롤러)
asyncio.gather() 기반, httpx + Playwright 이중 수집

→

STEP 3
가공·저장
LLM 한국어 요약·분류 후 PostgreSQL 저장

→

STEP 4
서비스 제공
Spring Boot API · React 웹

SERVERLESS
서버리스로 운영

EventBridge 스케줄이 AWS Lambda의 크롤러를 실행합니다.
JS 렌더링이 필요한 사이트는 Playwright 경로로, 나머지는 httpx로 가볍게 수집합니다.

Native Image로 시작 시간과 메모리 절감

Spring Boot를 GraalVM으로 AOT 컴파일해 배포하고, 같은 환경에서 JVM과 리소스 사용량을 비교했습니다.

시작 시간

0.457s

JVM 4.504s 대비
약 10배 단축

유휴 메모리

230MB

JVM 476MB 대비
약 52% 절감

유휴 CPU

0.1core

JVM 0.4 core 대비
사용량 감소

측정 환경 — MacBook Pro M4, Docker 2 CPU · 2GB 제한. 콜드 스타트와 idle 기준이며, 부하 상황의 처리량은 따로 비교했습니다.

빌드는 통과해도 런타임에서 깨지는 지점들

Native Image는 빌드 시점에 도달 가능한 코드만 포함합니다(closed-world). 동적으로 접근하는 코드를 하나씩 찾아 채웠습니다.

영역	해결 방법
리플렉션 · 프록시 · 리소스 접근	Tracing Agent로 실제 API 경로를 실행해 접근 정보를 캡처하고 reachability-metadata 생성
메타데이터 경로 · ByteBuddy 충돌	groupId/artifactId 경로와 ServiceLoader 등록 항목을 검토해 정리
SpEL 동적 타입 참조	T(...) 표현식 호출을 Spring Bean 참조로 전환
Hibernate 프록시 · Redis/Jackson 직렬화	캐시 직렬화와 프록시 호환성 이슈를 정리하고 해결

실행 경로를 학습시켜 다시 빌드

JIT이 없는 Native Image는 부하가 걸리면 느려집니다. 실제 호출 패턴을 프로파일로 수집해 빌드에 반영했습니다.

01
계측 빌드
`--pgo-instrument`

→

02
트래픽 재현
k6 부하 스크립트

→

03
프로파일 추출
`default.iprof`

→

04
최적화 빌드
`--pgo=default.iprof`

프로파일 대상

Redis I/O · Jackson · JPA

목록·카테고리·트렌딩 API는 매 요청마다 Redis 캐시 조회와 Jackson 직렬화·역직렬화를 반복합니다. 검색 API는 캐시 없이 QueryDSL + JPA 전문 검색을 수행합니다. 이 경로들을 k6로 집중 호출해 프로파일을 수집했습니다.

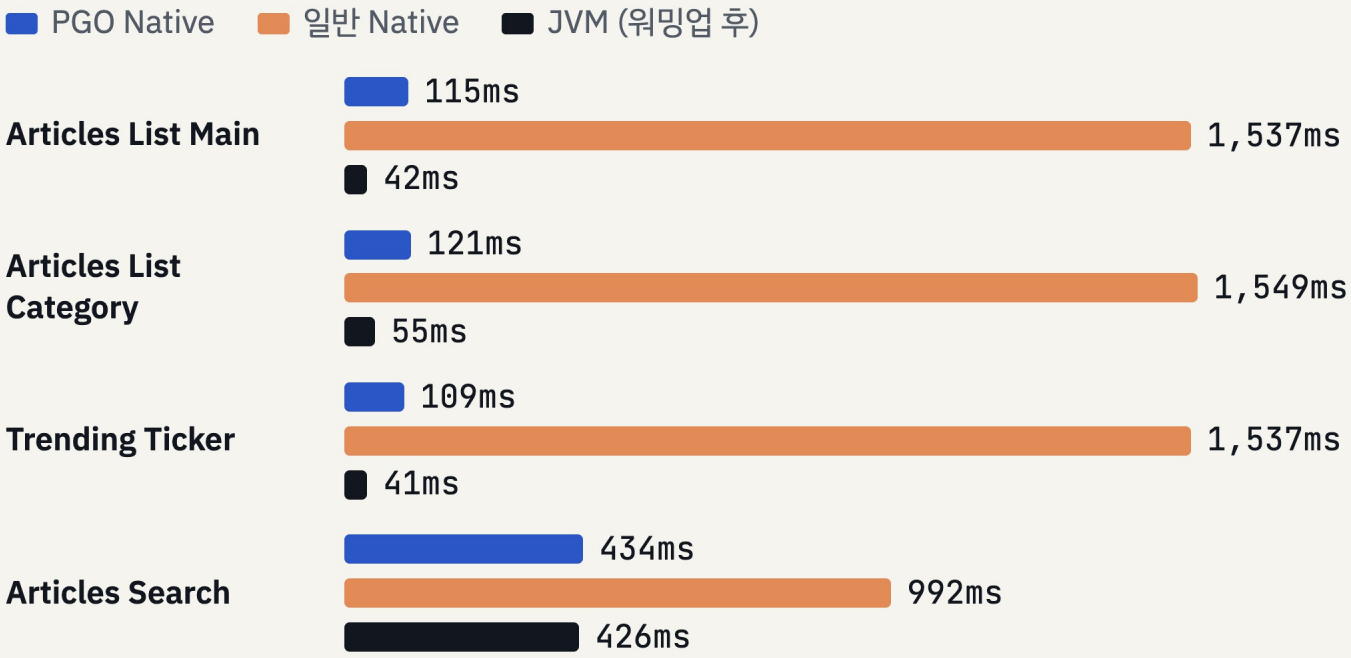
빌드 결과

최적화 레벨 O2 → O3

프로파일을 반영하자 Graal 컴파일러의 최적화 레벨이 한 단계 올라갔습니다. 계측 에이전트 때문에 808MB까지 커졌던 이미지는 가이드 빌드 후 275MB로 돌아왔습니다.

PGO로 주요 API p99 12.8–14.1배 개선

같은 읽기 중심 부하에서 일반 Native Image, PGO Native Image, 워밍업된 JVM의 p99 지연 시간을 비교했습니다.



비교 조건

2 vCPU · 2GB, 약 461 RPS 혼합 요청 6분. Nginx를 거치지 않고 앱 컨테이너에 직접 부하.

해석

충분히 워밍업한 JVM이 여전히 가장 빠릅니다. Native Image는 시작 시간·메모리를 얻는 대신 피크 성능을 양보하고, PGO로 그 격차를 줄였습니다.

01 · DEVPORT — ENGINEERING DECISION

별도 벡터 DB 대신 PostgreSQL + pgvector

위키 챗봇 하나를 위해 운영할 컴포넌트를 늘리기보다, 현재 요구사항을 기존 DB로 충족했습니다.

선택 이유

새 저장소를 추가하지 않고 기존 DB와 백업 체계를 그대로 활용

쓰기 부하 분리

위키를 개발 환경에서 생성·검증·임베딩한 뒤 프로덕션 DB로 이관해, 인덱스 쓰기 부하를 운영 트래픽과 떼어 놓았습니다.

트레이드오프

필터링과 하이브리드 검색은 직접 구현이 필요해 규모와 병목에 따라 전용 벡터 DB 전환을 검토해야 합니다.

01 · DEVPORT — RAG

의미와 키워드를 함께 찾는 위키 챗봇

두 검색 결과를 순위 기반으로 결합하고, LLM 리랭커로 문맥을 선별해 토큰 단위 스트리밍으로 답합니다.



컨텍스트 품질

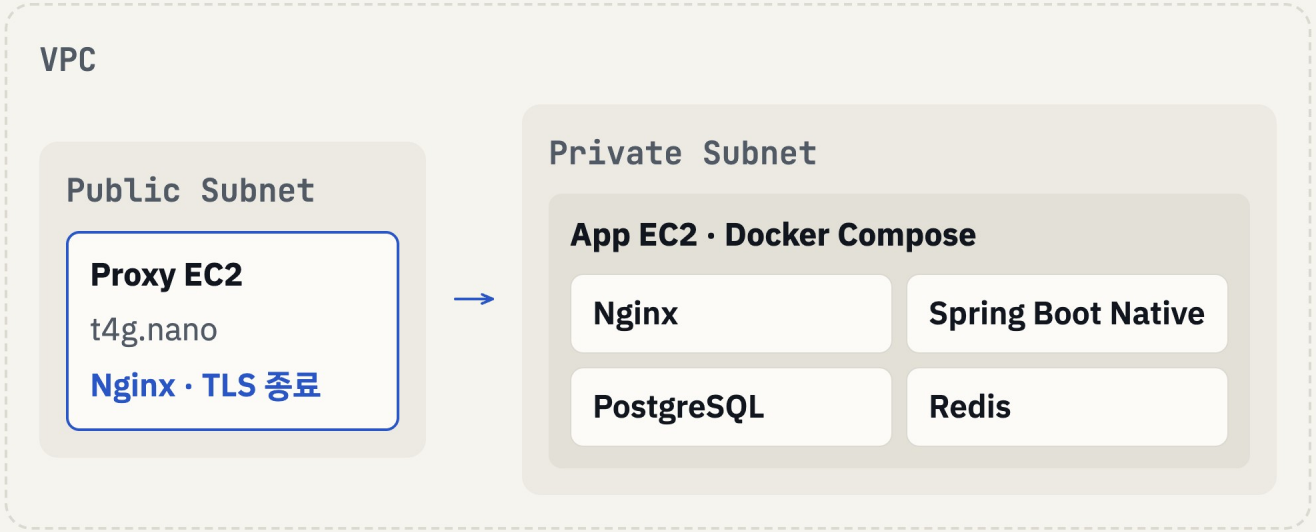
헤딩 겹침 제거 · 섹션 다양성 · FAQ 가중치 휴리스틱으로 최종 컨텍스트를 선별합니다.

응답 · 세션

Virtual Thread 기반 스트리밍, 대화 세션은 Redis(30분 TTL)와 DB 영속화로 이중 관리합니다.

혼자서도 운영할 수 있는 인프라와 배포

Terraform으로 환경을 재현 가능하게 만들고, 키 없는 배포 인증과 무중단 트래픽 전환을 자동화했습니다.



네트워크 · 복구

Public / Private Subnet 분리, 계층형 보안 그룹, Terraform State로 복구 체계 확보

Keyless CI/CD

GitHub Actions OIDC — 프론트엔드는 S3/CloudFront, 크롤러는 ECR로 배포

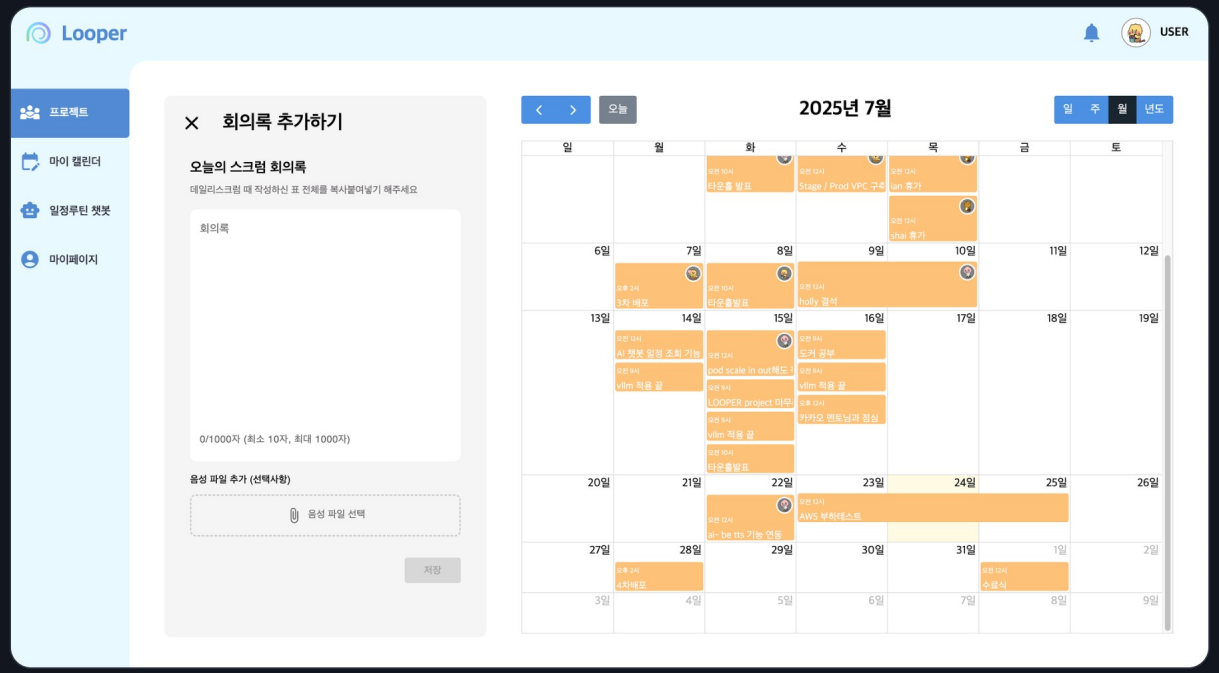
API Blue-Green

SSM 실행 → Green 기동·헬스체크 → Nginx 전환 → Blue 갱신

02 · LOOPER

AI 회의록 · 일정 관리 서비스의 인프라

카카오테크 부트캠프 6인 팀(DevOps 1 · Backend 1 · Frontend 1 · AI 3)에서 DevOps를 맡아 클라우드, 배포, 관측을 책임졌습니다.



해결하려던 문제

회의록 정리에 소요되는 시간과, 일정 수동 등록, 개인 일정과 팀 일정의 단절

내 역할

클라우드 IaC · kubeadm 클러스터 구축 · CI/CD · 모니터링과 분산 추적

기간

2025.04 – 2025.08 · 시연 영상

단일 VM에서 멀티 클라우드 클러스터까지

요구사항이 늘어날 때마다 단계적으로 확장하고, 모든 인프라를 Terraform으로 관리했습니다.

STAGE 1

단일 서버

VM 한 대에서 시작

STAGE 2

3-Tier

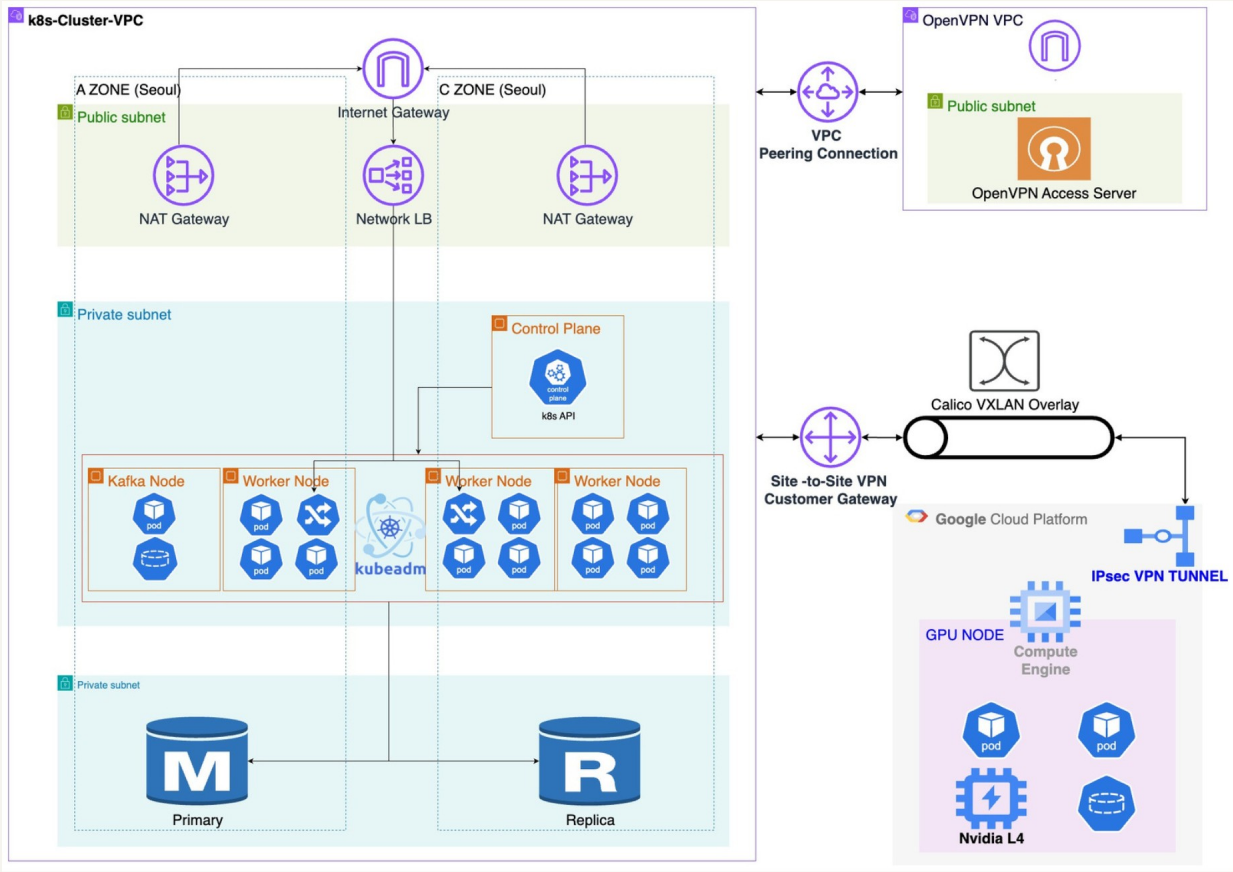
Docker 컨테이너로 계층 분리

STAGE 3

멀티 클라우드 클러스터

AWS + GCP를 묶은 kubeadm 클러스터

계층별 Subnet과 Security Group · NACL을 분리,
Private 인스턴스는 OpenVPN 터널로만 SSH 접근.



관리형 대신 kubeadm으로 직접 구축한 이유

두 클라우드에 흩어진 자원을 하나의 클러스터로 쓰는 것이 목표였습니다.

결정	이유
오케스트레이션 도입	서비스·미들웨어가 9개 이상에 모니터링 컴포넌트까지 늘며 Docker 단독 운영이 한계에 도달
kubeadm 직접 구축	크레딧 기반 GCP GPU 인스턴스를 AWS와 함께 쓰기 위해 — EKS · GKE는 각 클라우드에 종속
Calico CNI	노드 간 pod 네트워크를 오버레이로 연결 (IPIP에서 VXLAN으로 전환)
CRI-O	경량 컨테이너 런타임 채택
워크로드 배치	GPU 워크로드는 GCP, 그 외 워크로드는 AWS에 스케줄링

크로스 클라우드 pod 통신 장애: IPIP → VXLAN

AWS와 GCP 노드를 Site-to-Site VPN으로 연결한 뒤 발생한 문제입니다.

증상

노드간 ping은 되지만 pod간 통신은 안 됨

VPN 너머 노드 간 통신은 정상이지만 pod-to-pod 통신만 차단

→

원인

IPIP 패킷 드롭

VPN(Customer Gateway)은 TCP · UDP · ICMP 등만 통과. Calico 기본 오버레이인 IPIP의 inner 패킷이 드롭

→

해결

UDP 기반 VXLAN으로 전환

IPPool의 IPIP 의존성 확인 → VXLAN(UDP 4789) IPPool 재구성 → SG · NACL 규칙 갱신

→

결과

멀티 클라우드 pod 네트워크

GPU 워크로드는 GCP, 그 외는 AWS에 배치하는 크로스 클라우드 통신 구현

02 · LOOPER — CI/CD

GitOps 기반 배포 파이프라인

dev · staging · prod 브랜치 전략과 Kustomize 오버레이로 환경을 나누고, ArgoCD로 다운타임 없는 배포로 전환했습니다.



환경 분리

Kustomize Base + 환경별 patch, 네임스페이스별 리소스·변수

시크릿

External Secrets Operator + AWS Secrets Manager

이미지 태그

Semantic Release로 커밋 기반 버전 관리

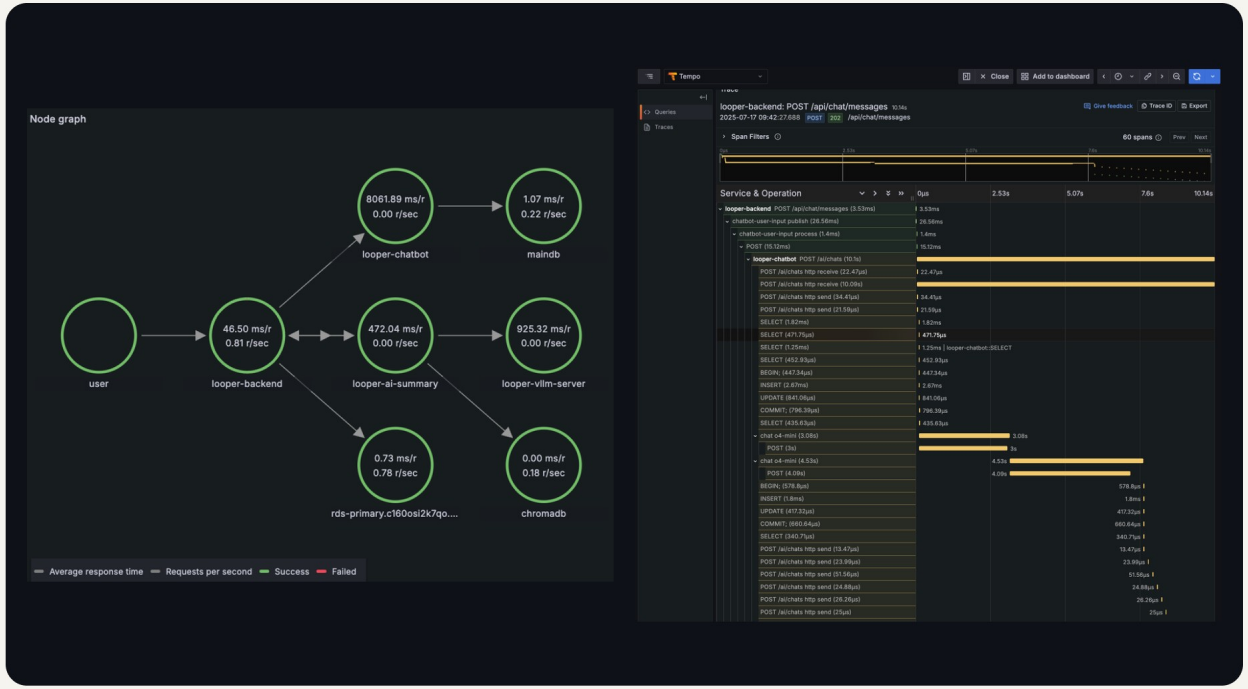
결과

배포 주 1~2회 → 하루 4~5회

배포 시간 약 60% 단축

분산 추적으로 찾은 두 개의 병목

Prometheus · Loki · Tempo 스택에 OpenTelemetry Collector를 붙여 Spring Boot와 FastAPI의 trace를 서비스 그래프로 시각화했습니다.



-60%

LLM 응답 시간

TraceQL로 OpenAI API 호출 지연의 약 70%가 프롬프트 구성 구간임을 식별하고 개선

150ms

평균 응답 (800ms에서)

쿼리 지연과 네트워크 병목을 찾아 Redis 캐싱 적용, 약 80% 개선

상태를 pod 밖으로: 세션과 대화 맥락

pod가 재시작되거나 늘어나도 사용자 경험이 끊기지 않도록, 상태를 저장하는 위치를 옮겼습니다.

SPRING BOOT

WebSocket(STOMP) 세션

문제

세션을 pod 메모리(ConcurrentHashMap)에만 보관 → pod 재시작 시 연결이 끊기고, 여러 pod로 확장할 수 없음

해결

연결 시 세션 ID와 사용자 정보를 Redis에 저장하고, 재시작 후 Redis에서 읽어 재연결. TTL로 메모리 관리

FASTAPI

챗봇 대화 맥락

문제

세션이 끊기면 멀티턴 대화의 슬롯 · 의도 · 상태가 유실

해결

PostgreSQL + asyncpg 풀에 히스토리와 메타데이터 저장. 마지막 ASSISTANT 메타데이터에서 상태를 복원하고, 스트리밍 종료 시 최종 상태와 서브태스크를 함께 저장

Sticky Session 대신 Redis를 택한 이유

특정 pod에 사용자를 묶는 방식은 그 pod가 사라지는 순간을 해결하지 못합니다.

기준	Redis 세션 저장 (채택)	Sticky Session
전제	이미 Cache-Aside 전략으로 Redis를 운영 중	Nginx Ingress Controller는 라우팅 전용으로 최소 리소스 유지 중
장점	기존 리소스 재사용, 추가 인프라 없음	L4 + L7 조합의 낮은 지연
단점	Redis 의존도 증가	Nginx 부하 · 리소스 증가, Nginx pod 의존도 증가
pod 재시작 시	세션 유지	고정된 pod와 함께 세션도 유실

03 · 반갈죽

제한된 자원에서 실시간 채팅 버티기

실시간 채팅 애플리케이션 부하테스트 토너먼트 · Backend · 2025.07 · Node.js / Redis / MongoDB

3위
22개 참가팀 중
2인 팀 · 본인 기여도 70%

30대
t3.small 인스턴스 한정
관리형 서비스 사용 금지

40+
pod 간 동기화
실시간 메시지 브로드캐스트

맡은 역할

Redis Cluster 구축과 연동, MongoDB 조회 경로 최적화, 클러스터 감지 · fallback 로직

핵심 문제

반복되는 aggregate 쿼리의 조회 지연, 인메모리 Socket.IO 어댑터의 확장 한계

03 · 반갈죽 — OPTIMIZATION

캐시 우선 조회와 Pub/Sub 메시지 전달

DB 쿼리 전에 캐시를 확인하고, Socket.IO 어댑터를 Redis Pub/Sub로 바꿔 pod 간 메시지를 동기화했습니다.

채팅방 목록 요청 → 캐시 확인 · ioredis + Redis Set 키 추적 → Cache miss → MongoDB → 캐시 갱신 · 응답

70ms

목록 조회 p99

145ms에서 약 51% 개선

-60%

DB 쿼리 수

캐시 hit ratio 90% 이상

95%+

자동 재연결 성공률

pod 장애 시

Socket.IO 인메모리 어댑터 → Redis Pub/Sub 어댑터 · 클러스터 모드 자동 감지와 fallback

SKILLS & BACKGROUND

개발을 통해 비즈니스 문제를 해결하는 개발자

기술을 선택한 이유와 실제 동작을 설명할 수 있는 개발을 지향합니다.

주력 기술

Java · Python · Spring Boot · JPA

PostgreSQL · AWS

교육

동국대학교 경영정보학 · 융합소프트웨어 (2019–2025)

카카오테크 부트캠프 클라우드 네이티브 (2025)

활용 경험

Kubernetes · Terraform · Nginx · Redis · React

GitHub Actions · ArgoCD · OpenTelemetry

자격 · 수상 · 어학

AWS Solutions Architect – Associate · E²GEE 메이커톤 2023 대상

TOEIC 975 · TOEIC Speaking 180 (AM)

bumsookim333@naver.com · github.com/BrianKim913 · medium.com/@bumsookim333